



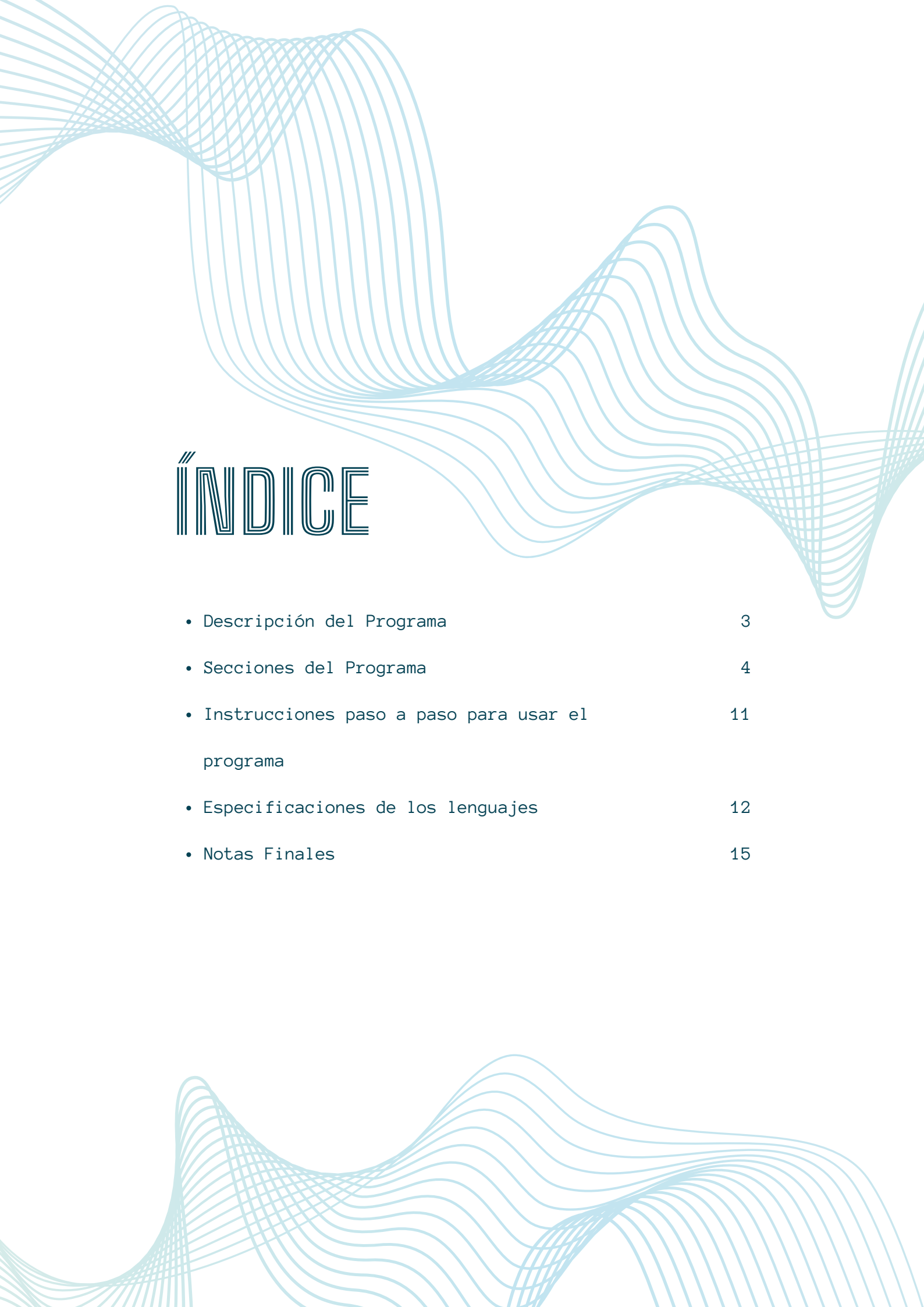
PROYECTO Q

CONTACTO

3XTRAT3RR3STR3D

MANUAL DE USUARIO

DAYANA SOFÍA OROZCO MENDÓZA 202231396



ÍNDICE

• Descripción del Programa	3
• Secciones del Programa	4
• Instrucciones paso a paso para usar el programa	11
• Especificaciones de los lenguajes	12
• Notas Finales	15

DESCRIPCIÓN DEL PROGRAMA

ExtraterrestreOS 3D es un entorno de desarrollo (IDE) de escritorio que permite escribir, validar y compilar programas escritos en tres lenguajes de alto nivel propios:

- Y? (.y) — define estructuras y funciones reutilizables (como una librería).
- Zetariano (.z) — define clases orientadas a objetos.
- Pig Latin (.pig) — es el lenguaje "principal": el punto de entrada del programa, que puede importar las estructuras/funciones de un .y y las clases de un .z.

El programa valida el código de forma léxica, sintáctica y semántica, lo traduce a código de tres direcciones (cuartetos), y finalmente genera y ejecuta código C equivalente — es decir, cubre todo el proceso desde el código fuente hasta un programa funcional.

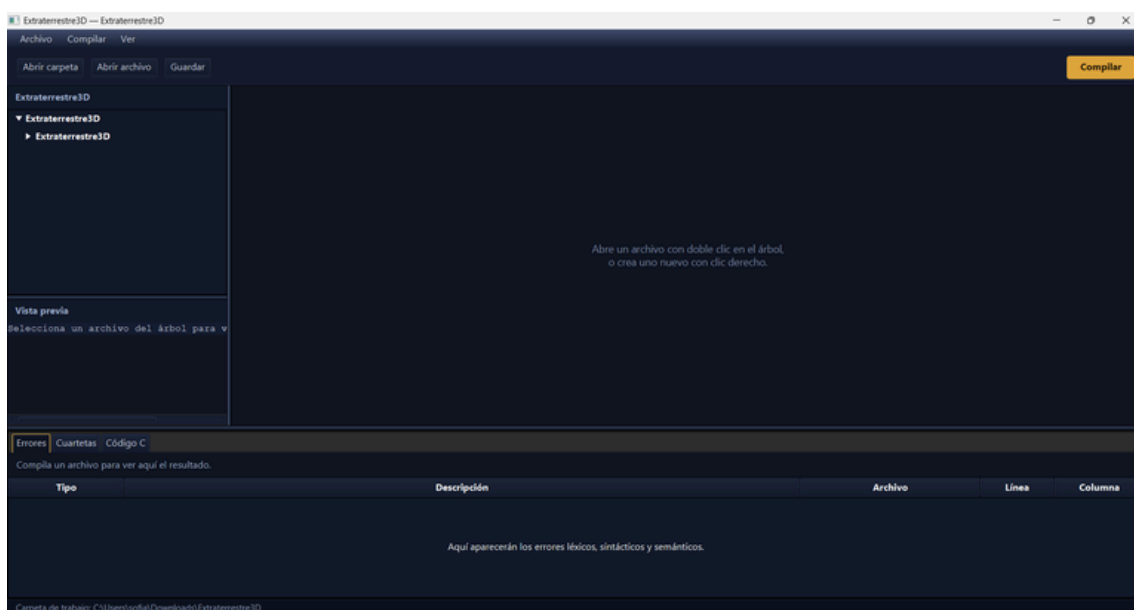
Requisitos para usarlo:

- Java 21 instalado (o el ejecutable/.jar ya empaquetado con Java incluido, según cómo te lo hayan entregado).
- Un compilador de C instalado y accesible desde la línea de comandos (gcc, clang o cc) — es necesario para que el botón Ejecutar de la pestaña "Código C" funcione.

SECCIONES DEL PROGRAMA

2.1 Pantalla principal

Al abrir el programa (o al abrir una carpeta de trabajo) se muestra la pantalla principal, dividida en 4 grandes zonas:

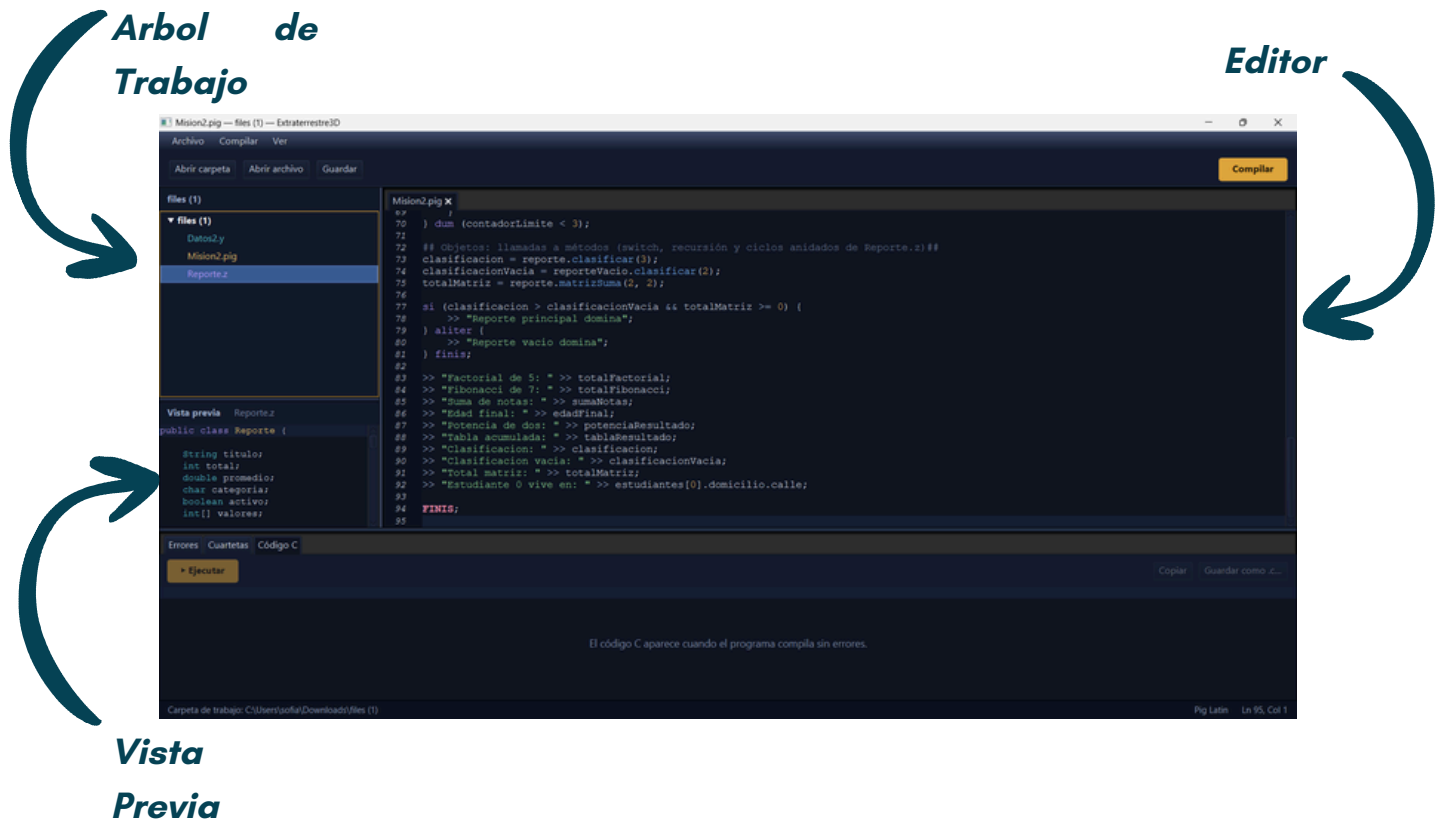


1. **Barra de menú** (Archivo, Compilar, Ver).
2. **Barra de herramientas:** botones Abrir carpeta, Abrir archivo, Guardar, y el botón Compilar (arriba a la derecha).
3. **Panel izquierdo:** el árbol de trabajo (arriba) y la vista previa del archivo seleccionado (abajo).
4. **Panel central:** el editor de código, con pestañas para varios archivos abiertos a la vez.
5. **Panel inferior:** resultados de la compilación, con tres pestañas — Errores, Cuartetas y Código C.

En la parte inferior de la ventana siempre se muestra la carpeta de trabajo actual (ej. Carpeta de trabajo: C:\Users\sofia\Downloads\Extraterrestre3D).

SECCIONES DEL PROGRAMA

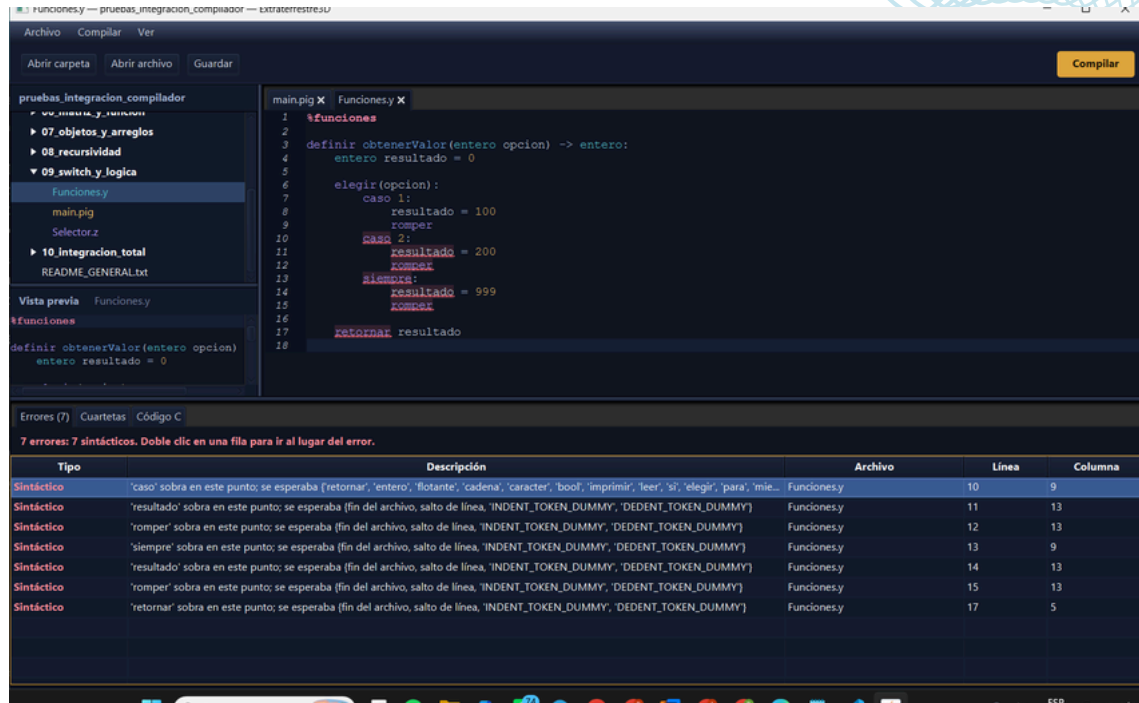
2.2 Árbol de trabajo, editor y vista previa



- **El árbol de trabajo** (panel superior izquierdo) muestra las carpetas y archivos de tu proyecto. Se puede navegar por subcarpetas (▶/▼) y abrir un archivo con doble clic, lo que lo agrega como una nueva pestaña en el editor.
- **El editor** (panel central) resalta la sintaxis de cada lenguaje en tiempo real (colores por tipo de token: palabras clave, tipos, literales, comentarios, etc.) y permite tener varios archivos abiertos en pestañas al mismo tiempo (.pig, .y, .z simultáneamente, como en la imagen).
- **La vista previa** (panel inferior izquierdo) muestra un resumen de solo lectura del archivo seleccionado en el árbol, sin necesidad de abrirlo como pestaña.

SECCIONES DEL PROGRAMA

2.3 Detección de errores



Al compilar, la pestaña Errores del panel inferior muestra una tabla con:

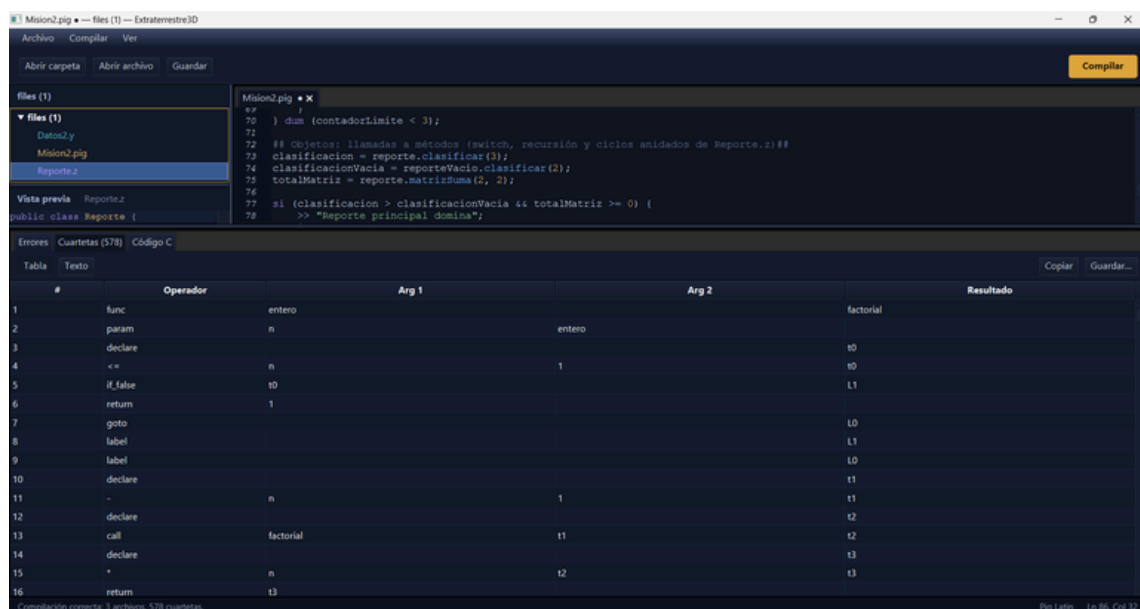
Columna	Contenido
Tipo	Léxico, Sintáctico o Semántico.
Descripción	El mensaje de error concreto (por ejemplo, qué token se encontró y cuáles se esperaban).
Archivo	En qué archivo del proyecto ocurrió (importante cuando el .pig importa .y/.z).
Línea / Columna	Ubicación exacta.

Doble clic en cualquier fila de error te lleva directo a esa línea y columna en el editor, así no tienes que buscarla manualmente.

SECCIONES DEL PROGRAMA

2.4 Pestaña "Cuartetas"

Una vez que el programa compila sin errores, esta pestaña muestra el código de tres direcciones generado, en una tabla con columnas #, Operador, Arg 1, Arg 2 y Resultado. Tiene dos vistas intercambiables (Tabla / Texto) y botones para Copiar el contenido o Guardar... las cuartetas en un archivo. Si el programa aún no ha compilado sin errores, la pestaña se muestra vacía con el mensaje "Las cuartetas aparecen cuando el programa compila sin errores."



#	Operador	Arg 1	Arg 2	Resultado
1	func	entero		factorial
2	param	n	entero	
3	declare			t0
4	<=	n	1	t0
5	if false	t0		L1
6	return	1		
7	goto			L0
8	label			L1
9	label			L0
10	declare			t1
11	-	n	1	t1
12	declare			t2
13	call	factorial	t1	t2
14	declare			t3
15	*	n	t2	t3
16	return	t3		



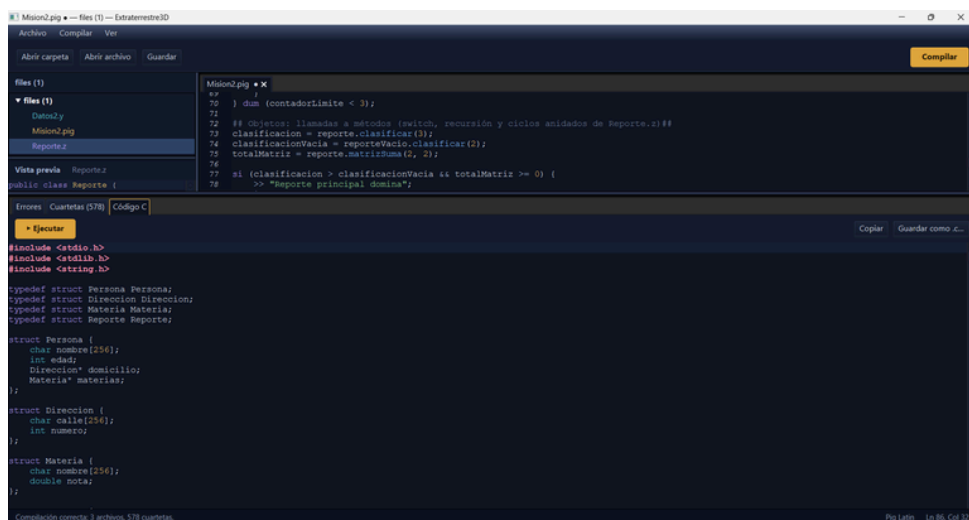
```
func factorial:
    param n
    declare BOOL t0
    t0 = n <= 1
    if false t0 goto L1
    return 1
    goto L0
L1:
L0:
    declare ENTERO t1
    t1 = n - 1
    declare ENTERO t2
    t2 = call factorial, t1
    declare ENTERO t3
    t3 = n * t2
    return t3
endfunc factorial
func fibonacci:
    param n
    declare BOOL t4
    t4 = n < 2
    if false t4 goto L3
    return n
    goto L2
L3:
L2:
    declare ENTERO t5
```

SECCIONES DEL PROGRAMA

2.5 Pestaña "Código C" y ejecución

Aquí se muestra el código C generado a partir de las cuartetas (el "bytecode preliminar ejecutable"). Incluye:

- El botón ► **Ejecutar**, que compila ese código C con el compilador instalado en tu equipo y corre el programa resultante.
- **Botones Copiar y Guardar como .c...** para exportar el código fuente en C.
- **Pestaña de ejecución:** al presionar Ejecutar, el programa muestra una consola/pestaña de ejecución con la salida real de tu programa (los imprimir/println/>> de tu código, y la entrada de teclado si tu programa la pide con leer/readln/<<). (Captura pendiente de agregar aquí — se añadirá una vez tomada.)



```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

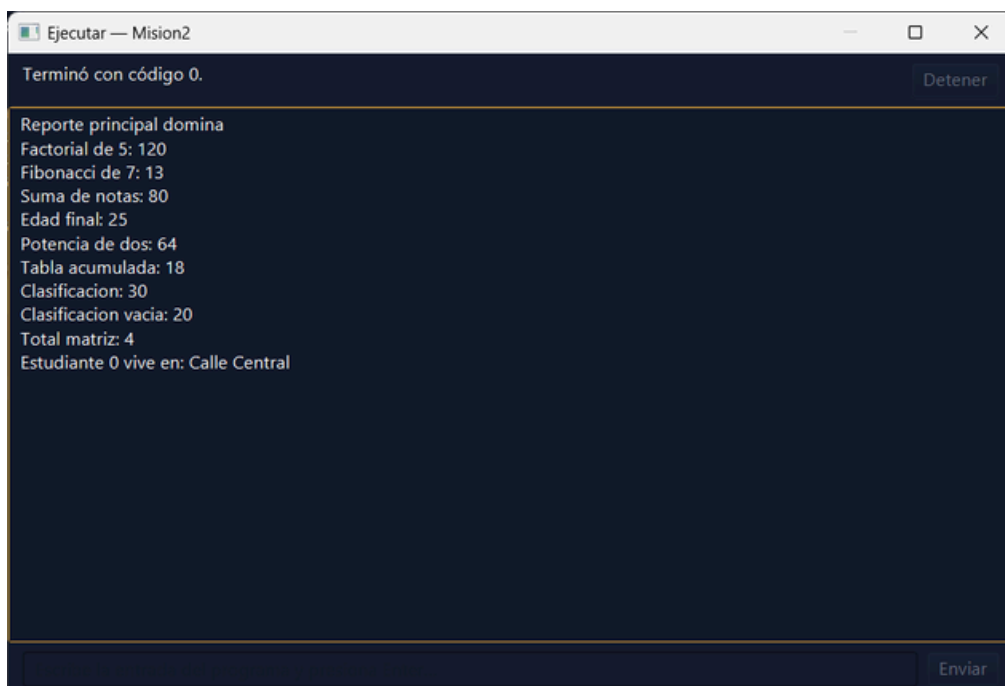
typedef struct Persona Persona;
typedef struct Direccion Direccion;
typedef struct Materia Materia;
typedef struct Reporte Reporte;

struct Persona {
    char nombre[256];
    int edad;
    Direccion* domicilio;
    Materia* materias;
};

struct Direccion {
    char calle[256];
    int numero;
};

struct Materia {
    char nombre[256];
    double nota;
};

// ... (rest of the code) ...
```

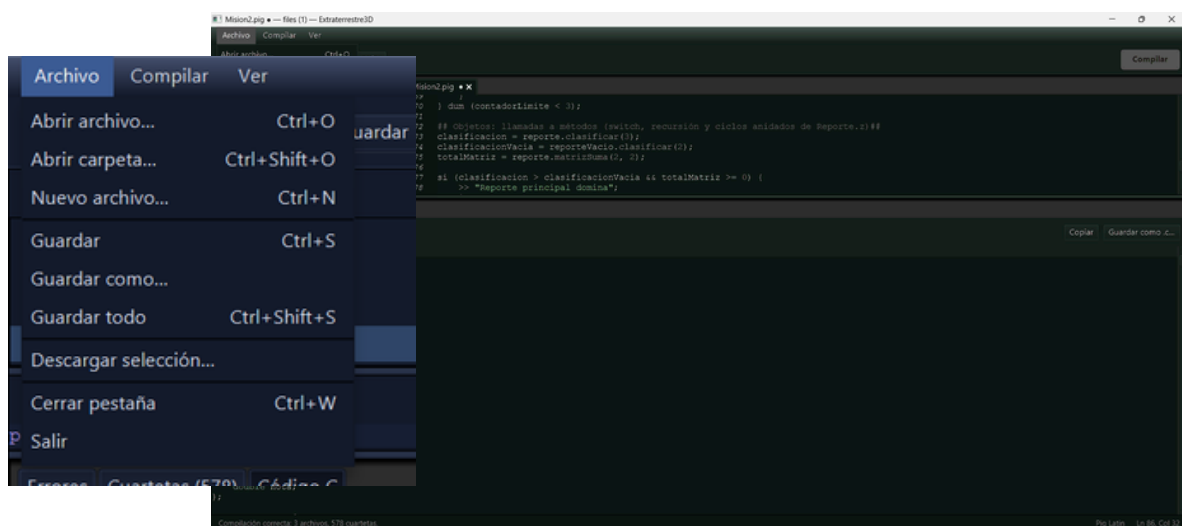


```
Terminó con código 0.

Reporte principal domina
Factorial de 5: 120
Fibonacci de 7: 13
Suma de notas: 80
Edad final: 25
Potencia de dos: 64
Tabla acumulada: 18
Clasificacion: 30
Clasificacion vacia: 20
Total matriz: 4
Estudiante 0 vive en: Calle Central
```

SECCIONES DEL PROGRAMA

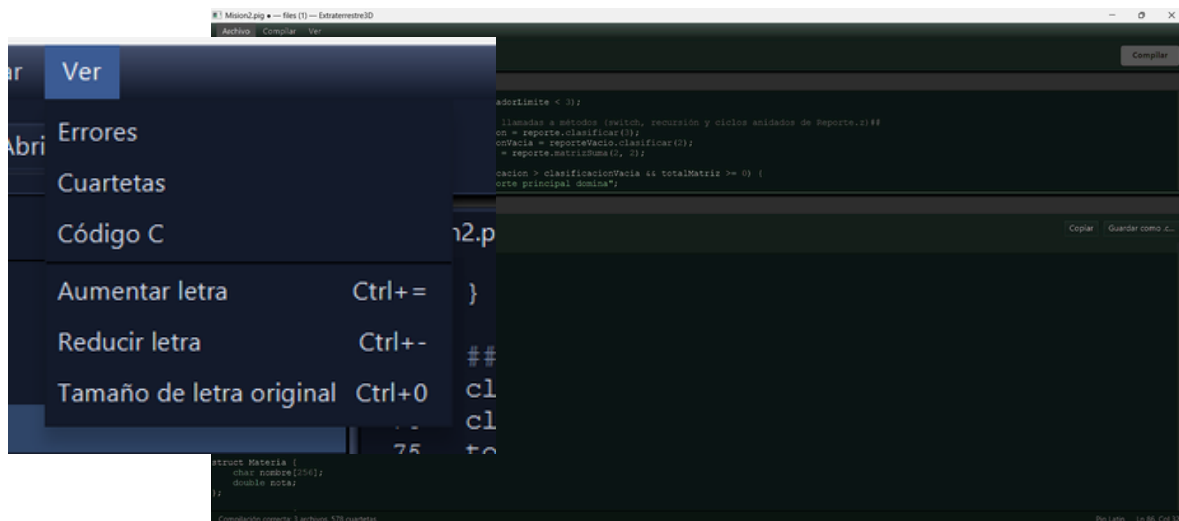
2.6 Menú "Archivo"



Opción	Atajo	Función
Abrir archivo...	Ctrl+O	Abre un único archivo (.pig, .y o .z) suelto, sin necesidad de abrir toda una carpeta.
Abrir carpeta...	Ctrl+Shift+O	Abre una carpeta completa como proyecto (aparece en el árbol de trabajo).
Nuevo archivo...	Ctrl+N	Crea un archivo nuevo dentro del proyecto actual.
Guardar	Ctrl+S	Guarda el archivo de la pestaña activa.
Guardar como...	—	Guarda el archivo de la pestaña activa con otro nombre/ubicación.
Guardar todo	Ctrl+Shift+S	Guarda todos los archivos abiertos con cambios sin guardar.
Descargar selección...	—	Descarga a tu equipo el archivo o carpeta seleccionada en el árbol.
Cerrar pestaña	Ctrl+W	Cierra la pestaña activa del editor.
Salir	—	Cierra el programa.

SECCIONES DEL PROGRAMA

2.7 Menú "Ver"



Opción	Atajo	Función
Errores	—	Cambia el panel inferior a la pestaña de errores.
Cuartetas	—	Cambia el panel inferior a la pestaña de cuartetas.
Código C	—	Cambia el panel inferior a la pestaña de código C.
Aumentar letra	Ctrl+=	Agranda el tamaño de fuente del editor.
Reducir letra	Ctrl+-	Reduce el tamaño de fuente del editor.
Tamaño de letra original	Ctrl+0	Restaura el tamaño de fuente por defecto.

2.8 Menú "Compilar"

Contiene la acción equivalente al botón amarillo Compilar de la barra de herramientas: analiza léxica, sintáctica y semánticamente el archivo .pig activo (junto con todo lo que importe), y si no hay errores, genera las cuartetas y el código C.

INSTRUCCIONES PASO A PASO PARA USAR EL PROGRAMA

1. **Abre el programa.** Si es la primera vez, verás la pantalla inicial sin ningún proyecto cargado (sección 2.1).
2. **Abre tu proyecto:** usa el botón Abrir carpeta de la barra de herramientas (o Archivo → Abrir carpeta..., Ctrl+Shift+O) y selecciona la carpeta que contiene tus archivos .pig, .y y .z. También puedes abrir un solo archivo suelto con Abrir archivo (Ctrl+O).
3. **Navega el árbol de trabajo** (panel izquierdo) y haz **doble clic sobre el archivo .pig** principal de tu programa para abrirlo en el editor.
4. **Escribe o edita tu código.** El editor resalta la sintaxis en tiempo real y subraya cualquier token que no encaje en la gramática mientras escribes.
5. **Guarda tus cambios** con Ctrl+S (o el botón Guardar) antes de compilar. Si tienes varios archivos abiertos con cambios, usa Guardar todo (Ctrl+Shift+S).
6. **Compila** presionando el botón amarillo **Compilar** (arriba a la derecha) o desde el menú Compilar.
7. **Revisa el resultado en el panel inferior:**
 - Si hay errores, aparecen en la pestaña Errores, con tipo, descripción, archivo, línea y columna. Haz doble clic en una fila para saltar directo a ese punto del código, corrígelo y vuelve a compilar.
 - Si no hay errores, revisa la pestaña Cuartetas para ver el código de tres direcciones generado.
8. **Ve o exporta el código C** en la pestaña **Código C:** puedes copiarlo, guardarlo como archivo .c, o presionar ► **Ejecutar** para compilarlo y correrlo directamente desde el programa.
9. **Observa la salida de tu programa** en la consola/pestaña de ejecución que se abre al presionar Ejecutar (ver nota en la sección 2.5).
10. Repite los pasos 4 a 9 las veces que necesites mientras desarrollas tu programa.

ESPECIFICACIONES DE LOS LENGUAJES

Esta sección es un resumen práctico para empezar a escribir código. El detalle exhaustivo de palabras reservadas, símbolos y gramática de cada lenguaje está en el Manual Técnico.

4.1 Y? (extensión .y) – estructuras y funciones

Se usa para declarar estructuras (datos) y funciones reutilizables que luego se importan desde un .pig. No admite variables globales. Es sensible a mayúsculas/minúsculas, y los bloques de código se definen por indentación (como en Python), no con llaves.

Tipos disponibles: entero, flotante, cadena, caracter, bool.

```
%estructuras
estructura Persona:
  cadena nombre
  entero edad

%funciones
definir saludar(entero edad) -> cadena:
  si(edad >= 18) entonces
    retornar "Adulto"
  retornar "Menor"
```

ESPECIFICACIONES DE LOS LENGUAJES

4.2 Zetariano (extensión .z) – clases

Lenguaje orientado a objetos con sintaxis similar a Java (llaves y punto y coma). El nombre del archivo debe ser igual al nombre de la clase que contiene (ej. la clase Persona debe estar en un archivo Persona.z).

Tipos disponibles: int, double, char, boolean, String.

```
public class Persona {  
    String nombre;  
    int edad;  
  
    public Persona(String nombreParam, int edadParam) {  
        nombre = nombreParam;  
        edad = edadParam;  
    }  
  
    public void saludar() {  
        println("Hola, soy " + nombre);  
    }  
}
```

ESPECIFICACIONES DE LOS LENGUAJES

4.3 Pig Latin (extensión .pig) – programa principal

Es el archivo con el que arranca tu programa. Importa estructuras/funciones (.y) y clases (.z), declara variables globales (opcional) y define la función principal. También es sensible a mayúsculas/minúsculas.

Tipos disponibles: numerus, textum, decimalis, littera (más los tipos importados desde .y/.z).

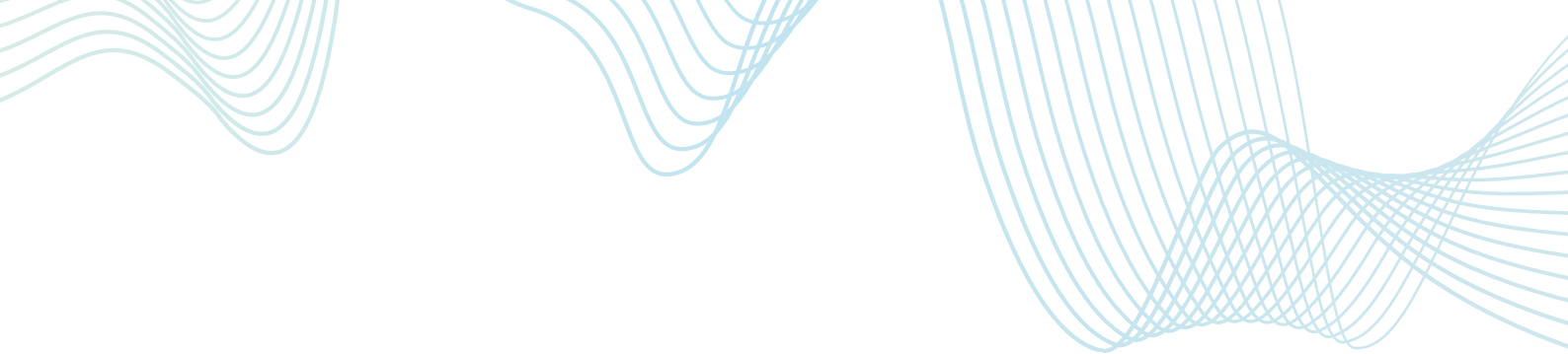
```
import Datos.y
import Persona.z

VARIABLES>
esto contador : numerus 0;
esto persona : Persona novus Persona("Ana", 20);

MAIOR>
per (esto i : numerus 0; i < 3; i++) {
    contador = contador + 1;
}
>> "Contador final: " >> contador;
FINIS;
```

Estructura general de un .pig:

```
import ...          // (opcional, una o más)
VARIABLES>          // (opcional)
    esto/series ...
MAIOR>
    ... sentencias ...
FINIS;
```



NOTAS FINALES

- Guarda siempre tus archivos antes de compilar; el compilador trabaja sobre lo que está guardado en disco.
 - Si el botón Ejecutar no funciona, verifica que tengas un compilador de C (gcc, clang o cc) instalado y accesible desde la terminal de tu sistema operativo.
 - Los errores se resuelven de arriba hacia abajo: como el análisis es léxico → sintáctico → semántico, a veces corregir el primer error de la lista hace que varios de los siguientes desaparezcan solos al volver a compilar.
- 